

**CENTRALE
LYON**

Modélisation Géométrique

BE : CRÉER UNE STRUCTURE DE DONNÉES DE MAILLAGE
EN C++

Étudiants :

LONDICHE Mathieu
DURAND Ulysse

Enseignant :

CHAINED Raphaëlle

11 février 2026

Table des matières

1	Implémentation	2
1.1	Maillage géométrique	2
1.1.1	Structure	2
1.1.2	Code	2
1.2	Maillage Topologique	3
1.2.1	Structure	3
1.2.2	Code	3
2	Example	4
2.1	Tétraèdre	4
2.2	Pyramide à base carré	5
2.3	Une boîte englobante 2D	7
3	Enregistrement et Visualisation du Maillage Géométrique	8
3.1	Code de la Fonction WriteOFF	8
3.2	Structure d'un Fichier .off	9
3.3	Visualisation des Maillage dans MeshLab	9
3.4	Chargement d'un fichier OFF en Maillage topologique	10

1 Implémentation

1.1 Maillage géométrique

1.1.1 Structure

$$\text{Tableau des points} = \begin{bmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ \vdots & \vdots & \vdots \\ x_N & y_N & z_N \end{bmatrix}$$

$$\text{Tableau des Faces} = \begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ \vdots & \vdots & \vdots \\ a_M & b_M & c_M \end{bmatrix}$$

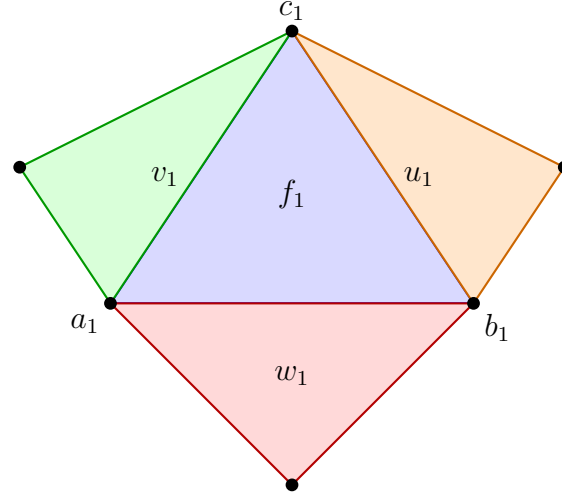
1.1.2 Code

```
1 #include <iostream>
2 #include <vector>
3
4 class Vertex {
5 public:
6     int x, y, z;
7
8     Vertex(int x_, int y_, int z_) : x(x_), y(y_), z(z_) {}
9 };
10
11 class Face {
12 public:
13     int a, b, c;
14
15     Face(int a_, int b_, int c_) : a(a_), b(b_), c(c_) {}
16 };
17
18
19 class Mesh {
20 public:
21     std::vector<Vertex> points;
22     std::vector<Face> faces;
23 };
```

1.2 Maillage Topologique

1.2.1 Structure

$$\text{Tableau des points} = \begin{bmatrix} x_1 & y_1 & z_1 & f_1 \\ x_2 & y_2 & z_2 & f_2 \\ \vdots & \vdots & \vdots & \vdots \\ x_N & y_N & z_N & f_N \end{bmatrix} \quad \text{Tableau des faces} = \begin{bmatrix} a_1 & b_1 & c_1 & u_1 & v_1 & w_1 \\ a_2 & b_2 & c_2 & u_2 & v_2 & w_2 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ a_M & b_M & c_M & u_M & v_M & w_M \end{bmatrix}$$



1.2.2 Code

```
1 #include <iostream>
2 #include <vector>
3
4 class Vertex_attached {
5 public:
6     int x, y, z, face;
7     Vertex_attached(int x, int y, int z, int face) :
8         x(x), y(y), z(z), face(face) {}
9 };
10 class Face_attached {
11 public:
12     int a, b, c, fa, fb, fc;
13     Face_attached(int a_, int b_, int c_, int fa_, int fb_, int fc_) :
14         a(a_), b(b_), c(c_), fa(fa_), fb(fb_), fc(fc_) {}
15 };
16 class Mesh_attached {
17 public:
18     std::vector<Vertex_attached> vertex;
19     std::vector<Face_attached> faces;
20 };
21 }
```

2 Example

2.1 Tétrahédre

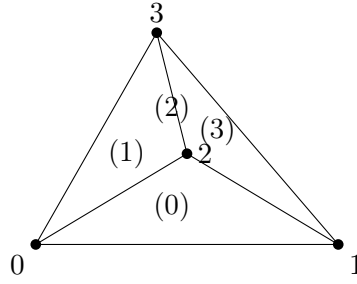


Tableau des sommets

$$P = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Tableau des faces

$$F = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 2 & 3 \\ 0 & 3 & 1 \\ 1 & 2 & 3 \end{bmatrix}$$

Tableau des sommets liés

$$P_{\text{lié}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Tableau des faces liées

$$F_{\text{lié}} = \begin{bmatrix} 0 & 1 & 2 & 3 & 1 & 2 \\ 0 & 2 & 3 & 3 & 4 & 2 \\ 0 & 3 & 1 & 3 & 1 & 4 \\ 1 & 2 & 3 & 1 & 2 & 4 \end{bmatrix}$$

```
1 Mesh mesh;
2
3 mesh.points.push_back(Vertex(0, 0, 0));
4 mesh.points.push_back(Vertex(1, 0, 0));
5 mesh.points.push_back(Vertex(0, 1, 0));
6 mesh.points.push_back(Vertex(0, 0, 1));
7 mesh.faces.push_back(Face(0, 1, 2));
8 mesh.faces.push_back(Face(0, 2, 3));
9 mesh.faces.push_back(Face(0, 3, 1));
10 mesh.faces.push_back(Face(1, 2, 3));
11
12 Mesh_attached mesh_attached;
13 mesh_attached.points.push_back(Vertex_attached(0, 0, 0, 0));
14 mesh_attached.points.push_back(Vertex_attached(1, 0, 0, 0));
15 mesh_attached.points.push_back(Vertex_attached(0, 1, 0, 0));
16 mesh_attached.points.push_back(Vertex_attached(0, 0, 0, 1));
17 mesh_attached.faces.push_back(Face_attached(0, 1, 2, 3, 1, 2));
18 mesh_attached.faces.push_back(Face_attached(0, 2, 3, 3, 4, 2));
19 mesh_attached.faces.push_back(Face_attached(0, 3, 1, 3, 1, 4));
20 mesh_attached.faces.push_back(Face_attached(1, 2, 3, 1, 2, 4));
```

2.2 Pyramide à base carré

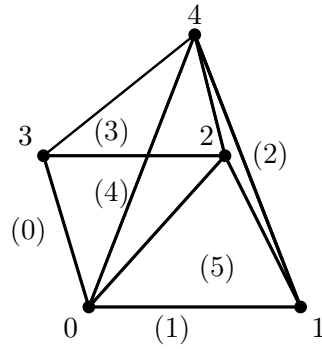


Tableau des sommets

$$P = \begin{bmatrix} 0 & 0 & 0 \\ 2 & 0 & 0 \\ 2 & 2 & 0 \\ 0 & 2 & 0 \\ 1 & 1 & 2 \end{bmatrix}$$

Tableau des faces

$$F = \begin{bmatrix} 0 & 4 & 3 \\ 0 & 1 & 4 \\ 1 & 2 & 4 \\ 2 & 3 & 4 \\ 0 & 3 & 2 \\ 0 & 2 & 1 \end{bmatrix}$$

Tableau des sommets liés

$$P_{\text{lié}} = \begin{bmatrix} 0 & 0 & 0 & 5 \\ 2 & 0 & 0 & 5 \\ 2 & 2 & 0 & 5 \\ 0 & 2 & 0 & 4 \\ 1 & 1 & 2 & 0 \end{bmatrix}$$

Tableau des faces liées

$$F_{\text{lié}} = \begin{bmatrix} 0 & 4 & 3 & 3 & 4 & 1 \\ 0 & 1 & 4 & 2 & 0 & 5 \\ 1 & 2 & 4 & 3 & 1 & 5 \\ 2 & 3 & 4 & 0 & 2 & 4 \\ 0 & 3 & 2 & 3 & 5 & 0 \\ 0 & 2 & 1 & 2 & 1 & 4 \end{bmatrix}$$

```

1  Mesh mesh;
2
3  mesh.points.push_back(Vertex(0, 0, 0));
4  mesh.points.push_back(Vertex(2, 0, 0));
5  mesh.points.push_back(Vertex(2, 2, 0));
6  mesh.points.push_back(Vertex(0, 2, 0));
7  mesh.points.push_back(Vertex(1, 1, 2));
8
9  mesh.faces.push_back(Face(0, 4, 3));
10 mesh.faces.push_back(Face(0, 1, 4));
11 mesh.faces.push_back(Face(1, 2, 4));
12 mesh.faces.push_back(Face(2, 3, 4));
13 mesh.faces.push_back(Face(0, 3, 2));
14 mesh.faces.push_back(Face(0, 2, 1));
15
16
17
18 Mesh_attached mesh_attached;
```

```
19
20     mesh_attached.points.push_back(Vertex_attached(0, 0, 0, 5));
21     mesh_attached.points.push_back(Vertex_attached(2, 0, 0, 5));
22     mesh_attached.points.push_back(Vertex_attached(2, 2, 0, 5));
23     mesh_attached.points.push_back(Vertex_attached(0, 2, 0, 4));
24     mesh_attached.points.push_back(Vertex_attached(1, 1, 2, 0));
25
26
27
28
29     mesh_attached.faces.push_back(Face_attached(0, 4, 3, 3, 4, 1));
30     mesh_attached.faces.push_back(Face_attached(0, 1, 4, 2, 0, 5));
31     mesh_attached.faces.push_back(Face_attached(1, 2, 4, 3, 1, 5));
32     mesh_attached.faces.push_back(Face_attached(2, 3, 4, 0, 2, 4));
33     mesh_attached.faces.push_back(Face_attached(0, 3, 2, 3, 5, 0));
34     mesh_attached.faces.push_back(Face_attached(0, 2, 1, 2, 1, 4));
```

2.3 Une boîte englobante 2D

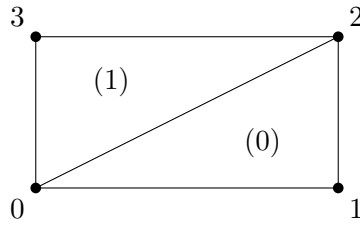


Tableau des sommets

$$P = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Tableau des faces

$$F = \begin{bmatrix} 0 & 1 & 2 \\ 0 & 2 & 3 \end{bmatrix}$$

Tableau des sommets liés

$$P_{\text{lié}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

Tableau des faces liées

$$F_{\text{lié}} = \begin{bmatrix} 0 & 1 & 2 & 1 & -1 & -1 \\ 0 & 2 & 3 & -1 & 0 & -1 \end{bmatrix}$$

```

1  Mesh mesh;
2
3  mesh.points.push_back(Vertex(0, 0, 0));
4  mesh.points.push_back(Vertex(0, 1, 0));
5  mesh.points.push_back(Vertex(1, 1, 0));
6  mesh.points.push_back(Vertex(1, 0, 0));
7
8
9
10 mesh.faces.push_back(Face(0, 1, 2));
11 mesh.faces.push_back(Face(0, 2, 3));
12
13 Mesh_attached mesh_attached;
14
15 mesh_attached.points.push_back(Vertex_attached(0, 0, 0, 0));
16 mesh_attached.points.push_back(Vertex_attached(0, 1, 0, 0));
17 mesh_attached.points.push_back(Vertex_attached(0, 1, 0, 1));
18 mesh_attached.points.push_back(Vertex_attached(1, 0, 0, 1));
19
20
21
22
23
24 mesh_attached.faces.push_back(Face_attached(0, 1, 2, 1, -1, -1));
25 mesh_attached.faces.push_back(Face_attached(0, 2, 3, -1, 0, -1));

```


3 Enregistrement et Visualisation du Maillage Géométrique

3.1 Code de la Foncion WriteOFF

```
1  #include <fstream>
2
3  bool writeOFF(const Mesh& mesh, const std::string& filename)
4  {
5      std::ofstream file(filename);
6      if (!file)
7      {
8          std::cerr << "Erreur : impossible d'ouvrir le fichier "
9                  << filename << std::endl;
10         return false;
11     }
12
13     // En-tête OFF
14     file << "OFF\n";
15     file << mesh.points.size() << " "
16         << mesh.faces.size() << " 0\n";
17
18     // Sommets
19     for (const auto& v : mesh.points)
20     {
21         file << v.x << " " << v.y << " " << v.z << "\n";
22     }
23
24     // Faces (triangles)
25     for (const auto& f : mesh.faces)
26     {
27         file << "3 "
28             << f.a << " "
29             << f.b << " "
30             << f.c << "\n";
31     }
32
33     return true;
34 }
```

3.2 Stucture d'un Fichier .off

```
1 OFF
2 5 6 0
3 0 0 0
4 2 0 0
5 2 2 0
6 0 2 0
7 1 1 2
8 3 0 4 3
9 3 0 1 4
10 3 1 2 4
11 3 2 3 4
12 3 0 3 2
13 3 0 2 1
```

3.3 Visualisation des Maillage dans MeshLab

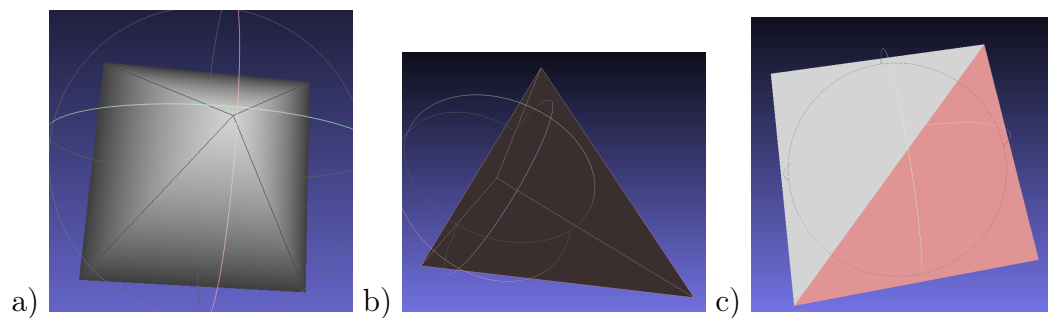


FIGURE 1 – a) Maillage Pyramidale, b) Maillage Tétrahédrique et c) Maillage 2D Box

3.4 Chargement d'un fichier OFF en Maillage topologique

La création manuelle d'un maillage est très vite compliquée, voici une fonction qui permet de créer c'st maillage automatiquement à partir d'un fichier .OFF

```
1      Mesh_attached loadOFF(const std::string& filename)
2  {
3      Mesh_attached mesh;
4
5      std::ifstream file(filename);
6      if (!file.is_open()) {
7          std::cerr << "Erreur : fichier non trouve\n";
8          return mesh;
9      }
10
11      std::string header;
12      file >> header;
13
14      int nbsommet, nbface, nbedge;
15      file >> nbsommet >> nbface >> nbedge;
16
17      // --- Lecture directe des sommets ---
18      mesh.points.reserve(nbsommet);
19      for (int i = 0; i < nbsommet; i++) {
20          float x, y, z;
21          file >> x >> y >> z;
22          mesh.points.emplace_back(x, y, z, -1); // face = -1 au debut
23      }
24
25      // --- Lecture directe des faces ---
26      mesh.faces.reserve(nbface);
27      for (int i = 0; i < nbface; i++) {
28          int skip, a, b, c;
29          file >> skip >> a >> b >> c;
30          mesh.faces.emplace_back(a, b, c, -1, -1, -1); // fa, fb, fc = -1
31      }
32
33      // --- Construction de la map ---
34      std::map<std::pair<int,int>, std::pair<int,int>>> edgeMap;
35
36      for (int f = 0; f < nbface; f++) {
37
38          Face_attached& F = mesh.faces[f];
39          int v[3] = {F.a, F.b, F.c};
40
41          for (int j = 0; j < 3; j++) {
42
43              int v1 = v[j];
44              int v2 = v[(j+1)%3];
45              int opposite = (j+2)%3;
46
47              // Attacher un face arbitraire au sommet
48              mesh.points[v1].face = f;
49          }
46      }
47  }
```

```

50     std::pair<int,int> e1 = {v1, v2};
51     std::pair<int,int> e2 = {v2, v1};
52
53     auto it1 = edgeMap.find(e1);
54     auto it2 = edgeMap.find(e2);
55
56     if (it1 != edgeMap.end() || it2 != edgeMap.end()) {
57
58         auto p = (it1 != edgeMap.end()) ? it1->second :
                    it2->second;
59
60         int f2 = p.first;
61         int opp2 = p.second;
62
63         // f <-> f2 adjacency
64         if (opposite == 0) F.fa = f2;
65         if (opposite == 1) F.fb = f2;
66         if (opposite == 2) F.fc = f2;
67
68         Face_attached& F2 = mesh.faces[f2];
69         if (opp2 == 0) F2.fa = f;
70         if (opp2 == 1) F2.fb = f;
71         if (opp2 == 2) F2.fc = f;
72     }
73     else {
74         edgeMap[e1] = {f, opposite};
75     }
76 }
77 }
78
79 return mesh;
80 }

```

Il ne manque plus que vérifier si le maillage retourner est identique au maillage calculer manuellement (on notera que indice de face associé à un point à plusieurs valeur possible comme il appartient à plusieurs face.

```

1      Mesh_attached test =loadOFF("pyramide.off");
2
3      std::cout << "Mesh charge :\n";
4      std::cout << "Nombre de Points : ";
5      std::cout << test.points.size() << "\n";
6
7      for (int j = 0; j < test.points.size(); j++) {
8          std::cout << "x : " << test.points[j].x
9                  << " | y : " << test.points[j].y
10                 << " | z : " << test.points[j].z
11                 << " | Face : " << test.points[j].face << "\n";
12      }
13
14      std::cout << "Nombre de Faces : ";
15      std::cout << test.faces.size() << "\n";
16
17      for (int j = 0; j < test.faces.size(); j++) {
18          std::cout << "a : " << test.faces[j].a
19                  << " | b : " << test.faces[j].b
20                  << " | c : " << test.faces[j].c
21                  << " | fa : " << test.faces[j].fa
22                  << " | fb : " << test.faces[j].fb
23                  << " | fc : " << test.faces[j].fc << "\n";
24      }

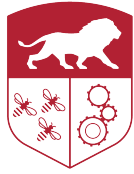
```

Une fois compilé le programme retourne :

```

1      Mesh charge :
2      Nombre de Points : 5
3      x : 0 | y : 0 | z : 0 | Face : 5
4      x : 2 | y : 0 | z : 0 | Face : 5
5      x : 2 | y : 2 | z : 0 | Face : 5
6      x : 0 | y : 2 | z : 0 | Face : 4
7      x : 1 | y : 1 | z : 2 | Face : 3
8      Nombre de Faces : 6
9      a : 0 | b : 4 | c : 3 | fa : 3 | fb : 4 | fc : 1
10     a : 0 | b : 1 | c : 4 | fa : 2 | fb : 0 | fc : 5
11     a : 1 | b : 2 | c : 4 | fa : 3 | fb : 1 | fc : 5
12     a : 2 | b : 3 | c : 4 | fa : 0 | fb : 2 | fc : 4
13     a : 0 | b : 3 | c : 2 | fa : 3 | fb : 5 | fc : 0
14     a : 0 | b : 2 | c : 1 | fa : 2 | fb : 1 | fc : 4

```



**CENTRALE
LYON**

Modélisation Géométrique

TP2 ET 3 – DIFFERENTIAL PROPERTIES ON SURFACES

Étudiants :

LONDICHE Mathieu

DURAND Ulysse

Enseignant :

CHAINED Raphaëlle

29 juillet 2026

Table des matières

1	Introduction	2
2	Calcul de Laplacien Propagation de chaleur	2
2.1	Calcul du Laplacien	2
2.2	1ère application : Diffusion de la chaleur	3
3	Affichage des Résultats	3
3.1	Structure du fichier	3
3.2	Échelle de couleur	4
4	Optimisation de l'algorithme	4
4.1	Stockage des valeurs redondantes	5
4.1.1	Structure des données	5
4.1.2	Empilage des vecteurs	5
4.1.3	Dépilage des vecteurs	6
4.2	Précalcul	6
4.3	Résolution explicite	7
4.4	Résultat	8
5	Courbure	8
6	Iterateurs et Circulateurs	10

1 Introduction

Les surfaces tridimensionnelles sont fréquemment représentées en informatique graphique et en géométrie algorithmique sous la forme de maillages triangulaires. Un maillage triangulaire est constitué d'un ensemble de sommets, d'arêtes et de faces triangulaires permettant d'approximer la géométrie d'une surface dans l'espace. Cette représentation discrète est largement utilisée pour l'analyse géométrique, la simulation physique ainsi que le traitement de formes.

Lors du premier TP, nous avons appris à construire un maillage géométrique à partir d'un fichier au format **OFF**. Dans cette structure de maillage, chaque sommet est décrit par quatre informations : ses coordonnées dans l'espace (x, y, z) ainsi qu'un indice de face auquel il appartient. Chaque face est quant à elle représentée par six informations : les indices des trois sommets qui la composent et les indices des trois faces voisines. Par ailleurs, une arête peut être identifiée à l'aide de deux informations : l'indice d'une face ainsi que l'indice relatif du sommet opposé à cette arête dans la face considérée.

Dans ce TP, nous nous intéressons au calcul du Laplacien discret d'une fonction définie sur les sommets du maillage à l'aide de la formule des cotangentes. Cet opérateur joue un rôle central en géométrie discrète et permet notamment de modéliser certains phénomènes physiques, comme la diffusion de la chaleur sur une surface.

Enfin, nous exploitons cet opérateur afin d'estimer certaines propriétés géométriques locales du maillage, telles que la normale et la courbure moyenne en chaque sommet. Ces informations peuvent ensuite être visualisées à l'aide d'un coloriage dépendant de la courbure, ce qui permet de mettre en évidence les caractéristiques géométriques de la surface étudiée.

2 Calcul de Laplacien Propagation de chaleur

2.1 Calcul du Laplacien

Sur une surface discrète représentée par un maillage triangulaire, les opérateurs différentiels doivent être adaptés à la structure discrète. Le Laplacien discret permet de mesurer la variation locale d'une fonction définie sur les sommets du maillage.

Soit une fonction scalaire u définie sur les sommets du maillage, où u_i représente la valeur de la fonction au sommet s_i . Le Laplacien discret peut être approximé à l'aide de la formule des cotangentes :

$$\Delta u(s_i) = \frac{1}{2A_i} \sum_{j \in N(i)} (\cot \alpha_{ij} + \cot \beta_{ij})(u_j - u_i)$$

où $N(i)$ est l'ensemble des voisins de s_i et A_i l'aire associée au sommet s_i . Cette formulation tient compte de la géométrie locale du maillage et est largement utilisée en géométrie discrète.

2.2 1ère application : Diffusion de la chaleur

Le Laplacien discret peut être utilisé pour simuler la diffusion de la chaleur sur une surface représentée par un maillage triangulaire. La température en chaque sommet du maillage est modélisée par une fonction $u(s, t)$ qui dépend du sommet s et du temps t . L'évolution de cette température est décrite par l'équation de la chaleur :

$$\frac{\partial u}{\partial t} = \Delta u$$

où Δu représente le Laplacien de la fonction u sur la surface.

Afin de résoudre cette équation numériquement, on utilise une discrétisation temporelle basée sur la méthode d'Euler explicite. Si u^t représente la température au temps t , l'évolution de la température au pas de temps suivant $t + \Delta t$ est donnée par :

$$u^{t+1} = u^t + \Delta t \Delta u^t$$

où Δt est le pas de temps choisi pour la simulation. Cette méthode permet ainsi de calculer itérativement la propagation de la chaleur sur l'ensemble des sommets du maillage.

Cette méthode présente l'avantage d'être simple à implémenter et peu coûteuse en calcul, car elle consiste uniquement à mettre à jour la température à chaque sommet à partir de la valeur du Laplacien au pas de temps courant.

Cependant, la méthode d'Euler explicite possède également certaines limites. En particulier, elle peut devenir instable si le pas de temps Δt est trop grand. Il est donc nécessaire de choisir un pas de temps suffisamment petit afin de garantir la stabilité et la convergence de la simulation.

Ainsi, il nous suffit à chaque itération de parcourir chaque voisin de chaque points du réseau, pour connaître l'évolution de la chaleur sur notre maillage.

Cette méthode peut être améliorée en utilisant par exemple la méthode d'Euler implicite ou le schéma de Crank–Nicolson. Ces méthodes permettent d'utiliser des pas de temps plus grands tout en garantissant une meilleure stabilité de la simulation, mais elles nécessitent en contrepartie la résolution d'un système linéaire à chaque itération. (inversion d'une Matrice $N \times N$).

3 Affichage des Résultats

Calculer les valeurs d'évolution de la chaleur, c'est intéressant, mais visualiser la variation de chaleur, c'est encore mieux. Pour cela, on garde le même format de fichier (`.off`) mais on y inclut le code couleur correspondant à la valeur affichée (température, courbure, ...).

3.1 Structure du fichier

Pour ajouter une couleur à une face, on rajoute le code RGB de la couleur après la définition de la face.

```

1 OFF
2 48339 96714 0
3 -0.0693915 0.0408189 0.144339
4 0.0691384 0.04012 0.144653
5 -0.0860709 0.0513071 -0.120459
6 0.0870232 0.0544774 -0.12014
7
8 . . .
9 . . .
10 3 15386 26862 26812 126 252 128 255 // Nb points, Point1, Point2
11 3 6926 24846 8415 126 252 128 255 // Point3, Rouge Vert Bleu
12 3 801 32595 23594 126 252 128 255
13
14 . . .
15 . . .

```

3.2 Échelle de couleur

Comme notre modèle comporte beaucoup de faces et que notre pas de temps est très petit, aux points les plus éloignés des sources de chaleur/froid, la variation de température est faible. On a donc choisi une échelle logarithmique pour mettre en évidence les plus petites variations.

```

1 inline std::array<unsigned char, 4> colormap(float t)
2 {
3     t = std::min(std::max(t, 0.0f), 1.0f);
4
5     float x = 2.0f * t - 1.0f;
6
7     float y = (x == 0.0f) ? 0.0f : std::copysign(std::log10(1.0f + 9.0f
8         * std::fabs(x)), x);
9
10    float c = 0.5f * (y + 1.0f);
11
12    unsigned char r = (unsigned char)(255 * c);
13    unsigned char g = (unsigned char)(255 * (1.0f - std::fabs(c - 0.5f)
14        * 2.0f));
15    unsigned char b = (unsigned char)(255 * (1.0f - c));
16    unsigned char a = 255;
17
18    return {r, g, b, a};
19 }
20 #endif

```

4 Optimisation de l'algorithme

On pourrait implémenter l'algorithme de manière naïve et tout recalculer pour chaque point à chaque itération. Cependant, cela entraîne une perte de temps importante. En

effet, notre maillage ne se déforme pas : les voisins d'un triangle restent toujours les mêmes et les triangles eux-mêmes ne changent pas. Par conséquent, les cotangentes associées aux angles des triangles et l'aires associée à chaque sommet restent également constantes.

4.1 Stockage des valeurs redondantes

Il suffit donc de rechercher les voisins et de calculer les cotangentes une seule fois au début du programme. Ces valeurs peuvent ensuite être stockées et réutilisées lors des calculs successifs.

4.1.1 Structure des données

Pour stocker les valeurs calculées, on crée deux vecteurs :

- un vecteur d'entiers qui stocke les indices des voisins ainsi qu'un indicateur marquant la fin de la liste des voisins ;
- un vecteur de flottants qui stocke les valeurs $\cot(\alpha)$, $\cot(\beta)$ ainsi que l'aire A associée.

Exemple de structure :

Voisins du point 0					Voisins du point N				
$v_{0,0}$	...	v_{0,n_0}	-1	...	-1	$v_{N,0}$	...	v_{N,n_N}	-1

Tableau d'entier

$\alpha_{0,0}$	$\beta_{0,0}$	...	α_{0,n_0}	β_{0,n_0}	A_0	...	A_{N-1}	$\alpha_{N,0}$	$\alpha_{N,0}$	...	α_{N,n_N}	α_{N,n_N}	A_N
----------------	---------------	-----	------------------	-----------------	-------	-----	-----------	----------------	----------------	-----	------------------	------------------	-------

Tableau de flottant

FIGURE 1 – Organisation du tableau d'entiers contenant les voisins.

4.1.2 Empilage des vecteurs

L'empilage des vecteurs se fait de manière similaire au calcul classique du laplacien. Après chaque calcul, on enregistre les valeurs obtenues dans les tableaux. Une fois tous les voisins d'un point parcourus, on ajoute un marqueur -1 dans le tableau d'entiers pour signaler la fin de la liste des voisins.

Ce marqueur permet également de savoir que l'on arrive à la fin des paires (α, β) correspondantes, ce qui permet de stocker l'aire A dans le même tableau que les cotangentes.

4.1.3 Dépilage des vecteurs

Pour calculer le laplacien en chaque point, il suffit alors de parcourir simultanément le tableau d'entiers et le tableau de flottants. Les opérations nécessaires sont donc :

- $(\text{len}(\text{tableau d'entiers}) - N_{\text{points}})$ additions et multiplications;
- N_{points} divisions;
- $\text{len}(\text{tableau d'entiers})$ tests conditionnels (`if`).

Ce qui est bien plus faible que la méthode naïve.

4.2 Précalcul

On peut précalculer les alpha, beta et aires dans un simple array moins couteux à traverser que les segments du maillage du fait que c'est un simple tableau.

```
1 struct AlphasBetasAreas
2 {
3     std::vector<int> index;
4     std::vector<float> AreaAlphaBeta;
5 };
6
7 class LaplacianComputer
8 {
9 public:
10     TriangleMesh &mesh;
11     AlphasBetasAreas precomp;
12     vector<float> u_laplacian;
13
14     LaplacianComputer(TriangleMesh &m)
15         : mesh(m),
16           precomp(preprocessAlphasBetas()),
17           u_laplacian(m.vertices.size(), 0.0f) {}
18
19     AlphasBetasAreas preprocessAlphasBetas();
20
21     const vector<float> &laplacian(const vector<float> &u);
22 };
```

4.3 Résolution explicite

Pour la résolution explicite de l'équation différentielle on crée une classe HeatSimulator

```
1 class HeatSimulator
2 {
3 public:
4     float t;
5     map<int, float> dirichlet_boundary_condition;
6     LaplacianComputer lapl_comp;
7     int timestep = 0;
8
9     vector<float> u;
10
11 void step(float dt)
12 {
13     const auto &lap = lapl_comp.laplacian(u);
14     for (const auto &pair : dirichlet_boundary_condition)
15         u[pair.first] = pair.second;
16     for (int i = 0; i < u.size(); i++)
17         u[i] += lap[i] * dt;
18     for (const auto &pair : dirichlet_boundary_condition)
19         u[pair.first] = pair.second;
20     t += dt;
21     timestep++;
22 }
23
24 HeatSimulator(TriangleMesh &m, map<int, float> dir_cond)
25     : t(0),
26       dirichlet_boundary_condition(dir_cond),
27       lapl_comp(m),
28       u(m.vertices.size()) {}
29 };
```

Alors on peut lancer la simulation

```
1 void exp_heat_source(TriangleMesh &mesh, map<int, float> bound_cond,
2 string out_dir)
3 {
4     float dt = 1e-7;
5     HeatSimulator simu(mesh, bound_cond);
6     for (int i = 0; i < 1000000; i++)
7     {
8         simu.step(dt);
9         if (i % 5000 == 0)
10         {
11             mesh.writeFunctionOFF(simu.u, out_dir +
12 to_string(simu.timestep) + ".off");
13         }
14     }
15 }
```

4.4 Résultat

Pour visualiser l'évolution de la propagation de la chaleur, nous avons choisi d'écrire un fichier .off toutes les 5000 itérations, et ce, sur un total d'un million d'itérations, étant donné que le pas de temps est très petit. Les images intermédiaires sont présentées ci-dessous. Une échelle logarithmique a été utilisée, et nous avons introduit un point froid à -15 et un point chaud à $+15$.

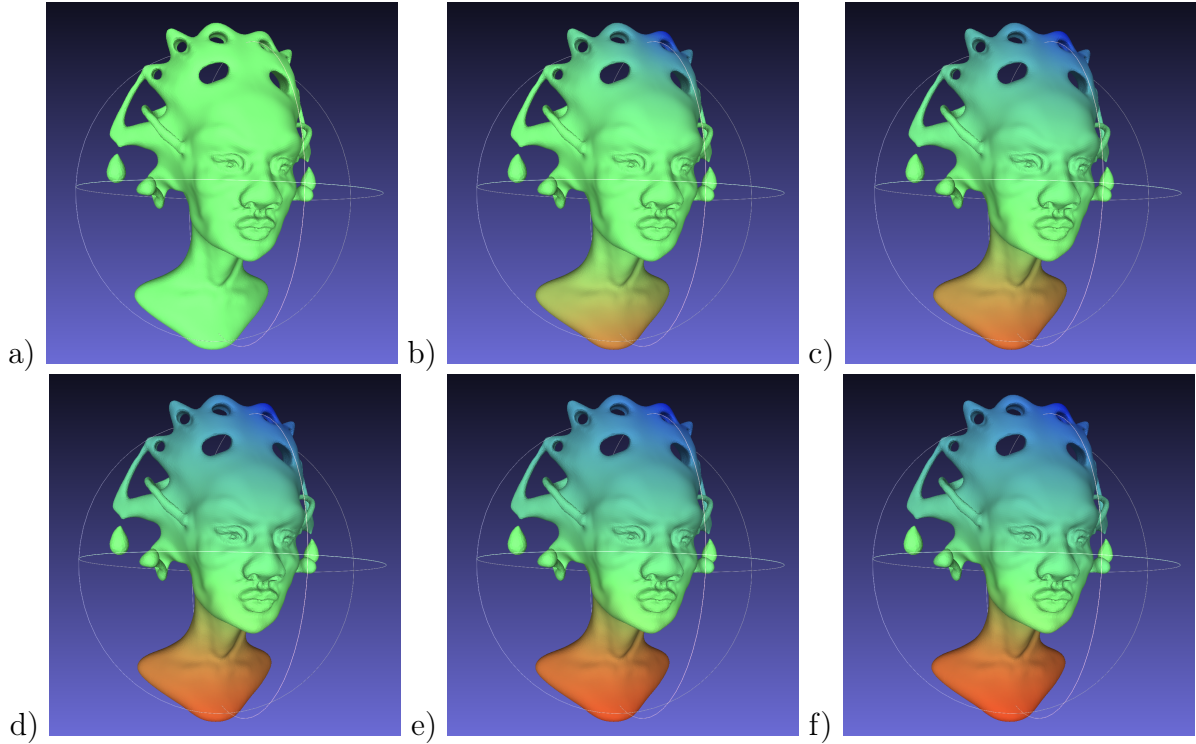


FIGURE 2 – a) 1 iterations, b) 200 000 itérations, c) 400 000 itérations, d) 600 000 itérations, e) 800 000 itérations et f) 1 000 000 itérations

5 Courbure

Courbure moyenne et matrice Laplacienne

On considère une surface discrète définie par $u(x, y)$. Le vecteur normal unitaire est :

$$\mathbf{n} = \frac{(-\partial_x u, -\partial_y u, 1)}{\sqrt{(\partial_x u)^2 + (\partial_y u)^2 + 1}}.$$

La courbure moyenne s est donnée par :

$$s = -\frac{1}{2} \nabla \cdot \mathbf{n}.$$

Pour des petites pentes ($|\partial_x u|, |\partial_y u| \ll 1$), on peut approximer :

$$s \approx -\frac{1}{2}\Delta u.$$

Le code de calcul de courbure est le suivant

```

1 void exp_curvature(TriangleMesh &mesh, string out_dir)
2 {
3     vector<float> ux(mesh.vertices.size(), 0.0f);
4     vector<float> uy(mesh.vertices.size(), 0.0f);
5     vector<float> uz(mesh.vertices.size(), 0.0f);
6     for (Iterator_on_vertices i = mesh.vertices_begin(); i !=
7         mesh.vertices_past_the_end(); ++i)
8     {
9         ux[*i] = mesh.vertices[*i].x;
10        uy[*i] = mesh.vertices[*i].y;
11        uz[*i] = mesh.vertices[*i].z;
12    }
13    LaplacianComputer lap_comp(mesh);
14    const vector<float> &lap_ux = lap_comp.laplacian(ux);
15    const vector<float> &lap_uy = lap_comp.laplacian(uy);
16    const vector<float> &lap_uz = lap_comp.laplacian(uz);
17
18    vector<Vector> lap_vec(mesh.vertices.size(), Vector(0, 0, 0));
19    vector<float> curv(mesh.vertices.size(), 0.0f);
20    for (Iterator_on_vertices i = mesh.vertices_begin(); i !=
21        mesh.vertices_past_the_end(); ++i)
22    {
23        lap_vec[*i] = Vector(lap_ux[*i], lap_uy[*i], lap_uz[*i]);
24        curv[*i] = max(0.0f, min(sqrt(norm(lap_vec[*i])), 20.0f));
25    }
26    mesh.writeFunctionOFF(curv, out_dir);
27 }

```

Le code permet d'obtenir le maillage illustré ci-dessous. L'affichage des vecteurs normaux n étant trop complexe, seul la courbure, est représentée.

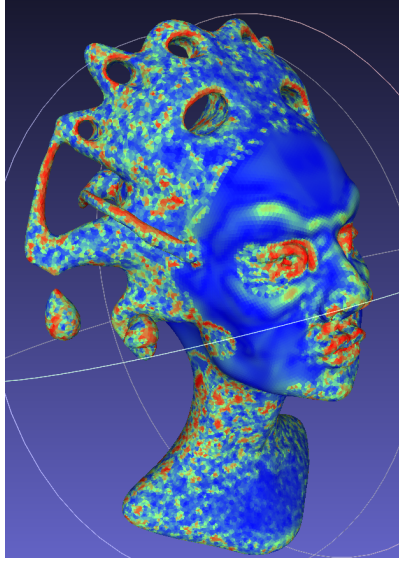


FIGURE 3 – Calcul de $\sqrt{|\log(s)|}$ (pour une meilleure visualisation de la courbure).

6 Iterateurs et Circulateurs

Le code précédent utilise au plus possible ces itérateurs et circulateurs.

Le programme à tester renvoie cette sortie :

```
1 valence of the vertex 3
2 valence of the vertex 4
3 valence of the vertex 3
4 valence of the vertex 4
5 valence of the vertex 4
6 valence of the vertex 3
7 valence of the vertex 4
8 valence of the vertex 3
```


Triangulation de Delauney

Ulysse DURAND, Mathieu LONDICHE ## Essayer le code

Il faut compiler et lancer `tp3.cpp`

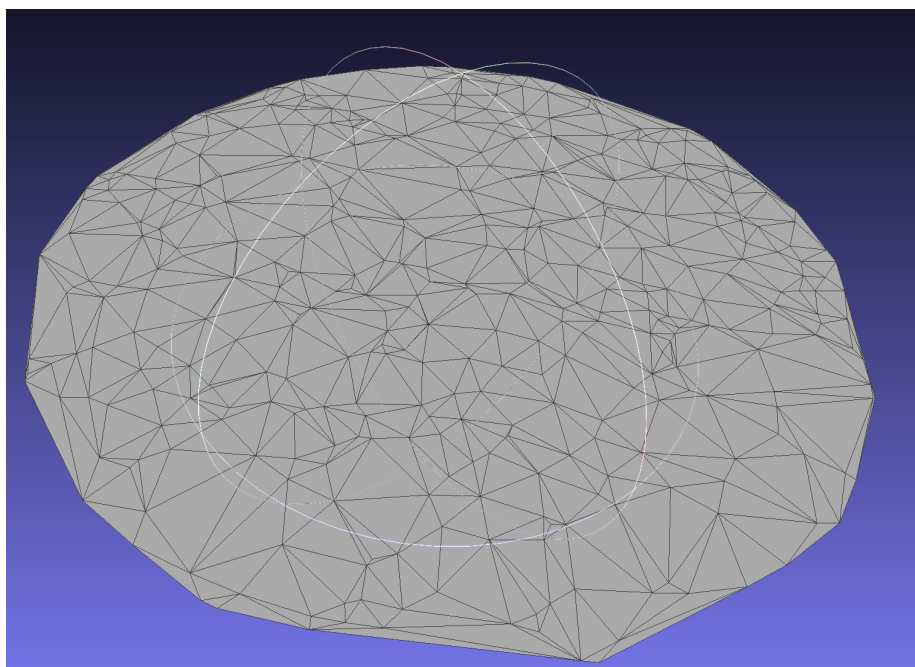
Le code

Les fonctions sont présentes dans `triangulation.h`.

La triangulation se fait avec la fonction `triangulize`.

Résultats

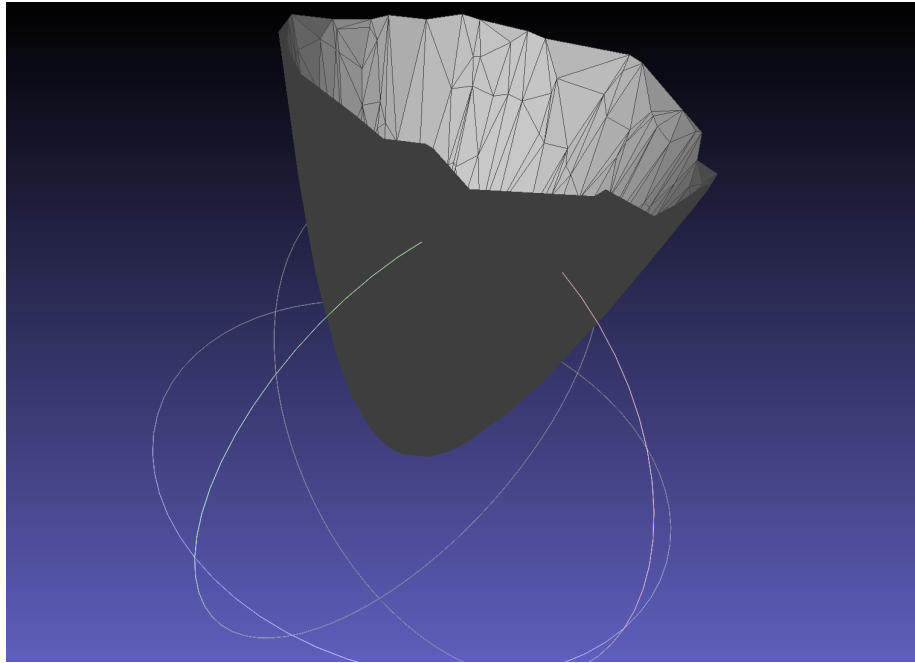
Ici nous avons la triangulation de 400 points mis au hasard dans un disque



Vérification

La fonction `checkValidityDelaunay` dans `triangulation.h` vérifie que le mesh généré est bien celui d'une triangulation de Delauney.

Nous pouvons difficilement vérifier que c'est de Delauney avec ce visualiseur mais nous ne voyons aucune partie concave.



Le mesh est disponible dans `triangulation.off`

Test de Complexité

Après plusieurs exécutions de l'algorithme sur la même machine, avec une nouvelle génération aléatoire de points à chaque lancement, nous avons réalisé un ajustement en échelle log-log des temps mesurés. La pente obtenue est d'environ **1.5**, ce qui correspond au comportement attendu : la recherche de la position du nouveau point dans le maillage s'effectue en $\mathcal{O}(\sqrt{N})$, tandis que les opérations suivantes sont en temps constant ou en temps constant amorti. A noter qu'à partir d'une trop grosse concentration de point, on a des erreurs dues à l'approximation des flottants.

Nous avons ici en abscisse le nombre de points à triangulariser et en ordonnée le temps en second qu'il a fallu pour faire la triangulation.

